

Comunicação

Desenvolvimento Web

Single-Page Applications

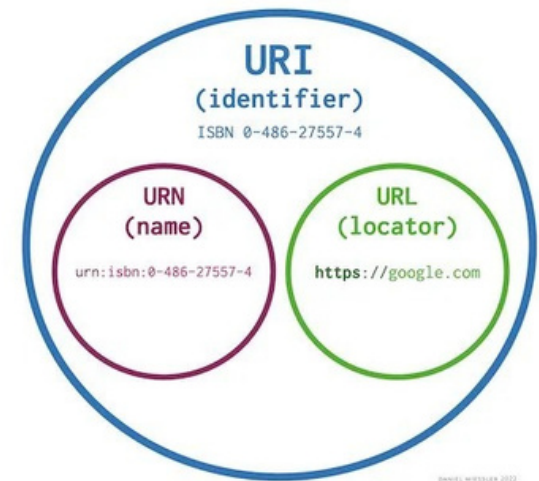
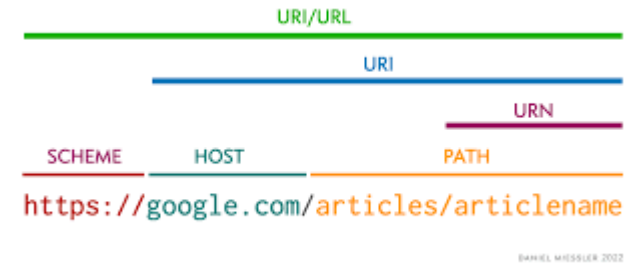
- A página web carrega mais dados por AJAX, normalmente ficheiros JSON, que usa para preencher templates client-side.
- Estes pedidos JSON devem ser estruturados numa API.
- Aplicações desktop e móveis necessitam de aceder a dados.
- Porque não usarem ambas uma API REST?

APIs RESTful

- REST é um estilo arquitetural para o desenvolvimento de **serviços na web**.
- REST é centrado em **recursos**, e existe um número fixo de operações sobre recursos.
- Permite vários tipos de formatos (JSON, XML, etc...)

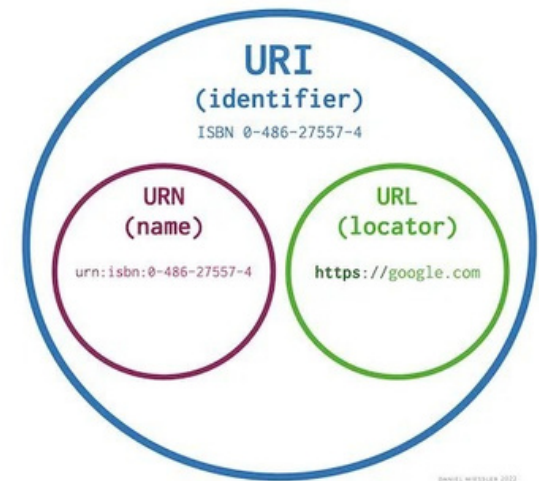
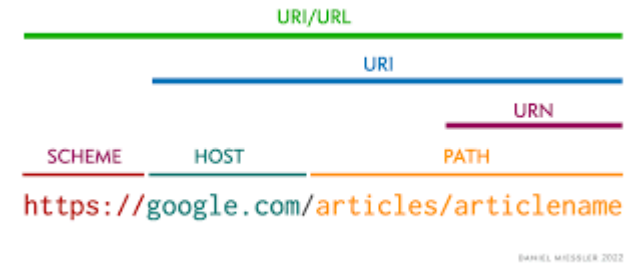
Princípios para APIs REST

- Interface uniforme
 - Recursos são identificados por URIs
 - Os recursos são descritos pelo servidor
 - Os clientes sabem o que podem alterar
 - Os clientes recebem recursos relacionados do servidor (hypermedia)



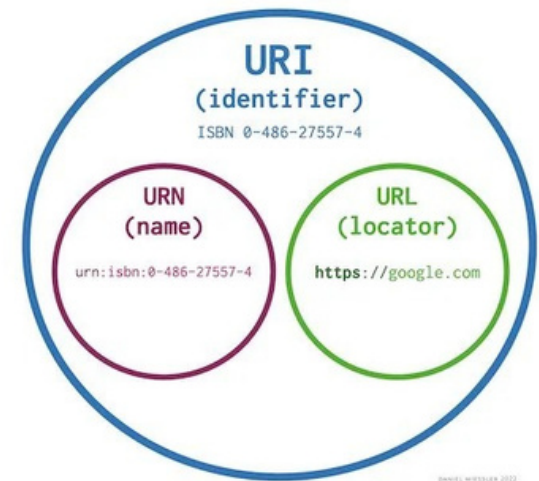
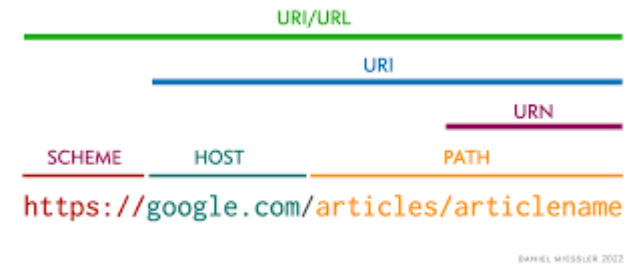
Princípios para APIs REST

- Interface uniforme
 - Recursos são identificados por URIs
 - Os recursos são descritos pelo servidor
 - Os clientes sabem o que podem alterar
 - Os clientes recebem recursos relacionados do servidor (hypermedia)
- Interface stateless
 - O cliente gere o estado, permitindo maior escalabilidade



Princípios para APIs REST

- Interface uniforme
 - Recursos são identificados por URIs
 - Os recursos são descritos pelo servidor
 - Os clientes sabem o que podem alterar
 - Os clientes recebem recursos relacionados do servidor (hypermedia)
- Interface stateless
 - O cliente gere o estado, permitindo maior escalabilidade
- Sistema em camadas
 - Camadas podem fornecer autenticação, autorização, cache, lógica de negócio, etc...



Pedido REST

GET	/pet/{petId}	Find pet by ID
POST	/pet/{petId}	Updates a pet in the store with form data
DELETE	/pet/{petId}	Deletes a pet

- Um pedido REST é um pedido HTTP
 - Tem verbo (GET, POST, PUT, DELETE, PATCH)
 - Tem um URL (que pode servir de URI)
 - Tem headers, e um corpo opcional (em caso de POST, PUT, PATCH)

Resposta REST

- Tem código (200 = OK)
- Tem corpo, que segue idealmente o formato pedido (header do pedido “accept”) e é descrito no header da resposta “content-type”). JSON é o mais comum (XML também), mas pode ser texto, imagem, etc...
- Tem headers com metadados.

Curl

```
curl -X 'GET' \
  'https://petstore.swagger.io/v2/pet/3' \
  -H 'accept: application/json'
```

Request URL

```
https://petstore.swagger.io/v2/pet/3
```

Server response

Code	Details
200	Response body <pre>{ "id": 3, "category": { "id": 3, "name": "sharks" }, "name": "buddy", "photoUrls": ["string"], "tags": [{ "id": 1, "name": "fish" }], "status": "available" }</pre> Response headers <pre>access-control-allow-headers: Content-Type,api_key,Authorization access-control-allow-methods: GET,POST,DELETE,PUT access-control-allow-origin: * content-type: application/json date: Tue, 23 May 2023 13:12:26 GMT server: Jetty(9.2.9.v20150224)</pre>

Responses

Verbos

Semântica

- GET /resource/id : obter uma representação do recurso
- POST /collection: adiciona o conteúdo do body como um novo elemento de collection
- PUT /resource/id: substitui o conteúdo desse recurso pelo que está a ser enviado no body (idempotente)
- PATCH /resource/id: altera o recurso parcialmente
- DELETE /resource/id: apaga o recurso
- Nota: Muitos developers confundem POST e PUT por causa do Create/Update, e não implementam a semântica correcta.

Hipermédia

Dados interligados

- Um recurso que contém outros sub-recursos devia tê-los indicados por URIs, de forma a serem descobertos pelos clientes.
 - Semelhante aos links em páginas web.
- Grande parte das implementações REST foge a esta regra, por motivos de performance: “31 é suficiente para representar `example.com/pet/31`”. A partir do ID chegamos ao URL pela documentação.

Hipermedia

Dados interligados

`getClient(123)`

Hipermedia

Dados interligados

getClient(123)

```
{
  "clienteId": 123,
  "nome": "João Silva",
  "empresa": "ABC Corp",
  "_links": {
    "self": { "href": "/clientes/123" },
    "pedidos": { "href": "/clientes/123/pedidos" },
    "editar": { "href": "/clientes/123" },
    "excluir": { "href": "/clientes/123" }
  }
}
```

```
{
  "clienteId": 123,
  "nome": "João Silva",
  "empresa": "ABC Corp",
  "pedidos": [ 501, 502, 503 ]
}
```

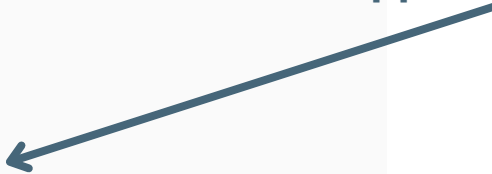
Hipermedia

Dados interligados

getClient(123)

Hypermedia as the Engine
of Application State

```
{
  "clienteId": 123,
  "nome": "João Silva",
  "empresa": "ABC Corp",
  "_links": {
    "self": { "href": "/clientes/123" },
    "pedidos": { "href": "/clientes/123/pedidos" },
    "editar": { "href": "/clientes/123" },
    "excluir": { "href": "/clientes/123" }
  }
}
```



```
{
  "clienteId": 123,
  "nome": "João Silva",
  "empresa": "ABC Corp",
  "pedidos": [ 501, 502, 503 ]
}
```

REST no Spring

Controllers

```
@RestController
@RequestMapping("/api/sales")
@Api(value = "Sale API", tags = "Sales")
public class SaleController {

    @GetMapping
    @ApiOperation(value = "Get all sales", notes = "Returns a list of all sales.")
    public ResponseEntity<List<SaleDto>> getAllSales() {
        return ResponseEntity.ok(saleHandler.getAllSales());
    }

    @GetMapping("/{saleId}")
    @ApiOperation(value = "Get sale by ID", notes = "Returns a sale by its ID.")
    public ResponseEntity<SaleDto> getSaleById(@PathVariable("saleId") long saleId) {
        SaleDto saleDto = saleHandler.getSaleById(saleId);
        return ResponseEntity.ok(saleDto);
    }
}
```

REST no Spring

Controllers

```
@RestController
@RequestMapping("/api/sales")
@Api(value = "Sale API", tags = "Sales")
public class SaleController {

    @GetMapping
    @ApiOperation(value = "Get all sales", notes = "Returns a list of all sales.")
    public ResponseEntity<List<SaleDto>> getAllSales() {
        return ResponseEntity.ok(saleHandler.getAllSales());
    }

    @GetMapping("/{saleId}")
    @ApiOperation(value = "Get sale by ID", notes = "Returns a sale by its ID.")
    public ResponseEntity<SaleDto> getSaleById(@PathVariable("saleId") long saleId) {
        SaleDto saleDto = saleHandler.getSaleById(saleId);
        return ResponseEntity.ok(saleDto);
    }
}
```

Controllers REST devolvem recursos que são serializados em vários formatos, nomeadamente JSON por omissão. É como desenvolver para Web, mas sem o HTML.

DTOs definem subsets do recurso para serem apresentados.

Acoplamento entre camadas

- Evitar expor campos internos da base de dados
 - `passwordHash`, `createdAt`, `internalId`
- Se a estrutura da base de dados mudar, a API não deve quebrar automaticamente
- A API pode precisar de apenas de 3 campos, mas a entidade tem 30 colunas. Deve se enviar o mínimo.
- Transformar dados complexos em formatos amigáveis para o cliente
 - Combinar campos, converter tipos

Data Transfer Objects

- Objeto simples, focado exclusivamente no transporte de dados entre processos ou camadas
- Não contém lógica de negócio apenas atributos e métodos de acesso
 - getters/setters ou records
- Funciona como um contrato
 - Define o que esperar entre do servidor por quem consome a API
- Separa a representação externa das entidades

Como usar DTOs

- Objetos imutáveis (e.g., Records em Java ou classes read-only) são uma boa opção
- *MapStruct* ou *ModelMapper* para converter entre Entities e DTOs
 - Biblioteca manual “Mapper” também funciona
- Anotações de validação (*@NotNull*, *@Size*, etc.) nos DTOs garantem que os dados do RequestBody estão corretos antes de chegarem ao serviço

REST no Spring

Controllers

```
@RestController
@RequestMapping("/api/sales")
@Api(value = "Sale API", tags = "Sales")
public class SaleController {

    @GetMapping
    @ApiOperation(value = "Get all sales", notes = "Returns a list of all sales.")
    public ResponseEntity<List<SaleDto>> getAllSales() {
        return ResponseEntity.ok(saleHandler.getAllSales());
    }

    @GetMapping("/{saleId}")
    @ApiOperation(value = "Get sale by ID", notes = "Returns a sale by its ID.")
    public ResponseEntity<SaleDto> getSaleById(@PathVariable("saleId") long saleId) {
        SaleDto saleDto = saleHandler.getSaleById(saleId);
        return ResponseEntity.ok(saleDto);
    }
}
```

Controllers REST devolvem recursos que são serializados em vários formatos, nomeadamente JSON por omissão. É como desenvolver para Web, mas sem o HTML.

DTOs definem subsets do recurso para serem apresentados.

REST no Spring

Controllers

```
@PatchMapping("/{saleId}/date")
@ApiOperation(value = "Update sale date", notes = "Updates the sale date.")
public ResponseEntity<SaleDto> updateSaleDate(
    @PathVariable("saleId") long saleId,
    @RequestBody String saleDate) {

    try {
        Optional<SaleDto> updatedSale = saleHandler.updateSaleDate(saleId, saleDate);
        if (updatedSale.isEmpty()) {
            return ResponseEntity.notFound().build();
        }
        return ResponseEntity.ok(updatedSale.get());
    } catch (DateTimeParseException e) {
        return ResponseEntity.badRequest().build();
    }
}
```

```
@DeleteMapping("/{saleId}")
@ApiOperation(value = "Delete sale", notes = "Deletes a sale by its ID.")
public void deleteSale(@PathVariable("saleId") long saleId) {
    saleHandler.deleteSale(saleId);
}
```

A resposta pode ser programaticamente alterada, incluindo usando o código e headers certos.

REST no Spring

Controllers

```
@PostMapping("/{saleId}/product")
@ApiOperation(value = "Add product to sale", notes = "Adds a product.")
public ResponseEntity<SaleDto> addProductToSale(
    @PathVariable("saleId") long saleId,
    @RequestBody SaleProductDto saleProd) {
    SaleDto saleDto = saleHandler.addProductToSale(saleId,
                                                    saleProd.getProductCode(),
                                                    saleProd.getQuantity());
    return ResponseEntity.ok(saleDto);
}
```

```
@PostMapping("/close/{saleId}")
@ApiOperation(value = "Close sale", notes = "Closes a sale")
public ResponseEntity<SaleDto> closeSale(@PathVariable("saleId") long saleId) {
    SaleDto saleDto = saleHandler.closeSale(saleId);
    return ResponseEntity.ok(saleDto);
}
```

REST no Spring

Controllers

```
@PostMapping("/close/{saleId}")
@ApiOperation(value = "Close sale", notes = "Closes a sale")
public ResponseEntity<SaleDto> closeSale(@PathVariable("saleId") long saleId) {
    SaleDto saleDto = saleHandler.closeSale(saleId);
    return ResponseEntity.ok(saleDto);
}

@PatchMapping("/{saleId}/status")
@ApiOperation(value = "Update sale status", notes = "Update to CLOSED or OPEN")
public ResponseEntity<SaleDto> updateSaleStatus(@PathVariable("saleId") long saleId,
    @RequestBody String status) {
    SaleDto saleDto;
    // ...
    switch (status.toLowerCase()) {
        case "close" -> saleDto = saleHandler.updateStatus(saleId, CLOSED);
        case "open" -> saleDto = saleHandler.updateStatus(saleId, OPEN);
        default -> {return ResponseEntity.badRequest().body(null);}
    }
    return ResponseEntity.ok(saleDto);
}
```

REST no Cliente

Para desenvolvimento

- CLI: Curl, HTTPie
- Desktop: Postman, Arc, CocoaRestClient, RESTClient
- Editor: Thunder Cliente (VSCode)
- Web: **Swagger**
- Java: HttpURLConnection (Java 9+)
- Python: requests

REST no Cliente

Para desenvolvimento

The screenshot shows the CocoaRestClient application window. The top bar contains three browser tabs: 'httpbin.org', 'httpbin.org', and 'google.com'. The main interface is divided into two main sections: 'REQUEST' and 'RESPONSE'.

REQUEST Section:

- URL:** `http://httpbin.org/put`
- Method:** `PUT`
- Submit** button
- Body:**

```
1 {  
2   "field one": "one",  
3   "field two": 2  
4 }
```
- Options:** ☒ Raw Input, ☐ Field Input/Multipart File Upload, ☐ Raw File Input

RESPONSE Section:

- Body:**

```
1 {  
2   "args": {  
3     },  
4   "data": "{\n  \"field one\": \"one\", \n  \"field two\": 2\n}",  
5   "files": {  
6     },  
7   "form": {  
8     },  
9   "headers": {  
10    "Accept": "application/json",  
11    "Accept-Encoding": "gzip, deflate",  
12    "Accept-Language": "en-us",  
13    "Connection": "close",  
14    "Content-Length": "46",  
15    "Host": "httpbin.org",  
16    "User-Agent": "CocoaRestClient/23 CFNetwork/811.4.18 Darwin/16.5.0 (x86_64)"
```

Saved Requests:

- ▼ cocoa-rest-client
 - appcast
 - releases
- ▼ httpbin
 - put**
 - redirect 6
 - stream 50
- ▼ auth
 - basic
 - failure
- ▼ cookies
 - getCookies
 - setCookie
 - delete

Finished in 0.089345 seconds

REST no Cliente

Para desenvolvimento

 **Swagger**
Powered by SMARTBEAR

Explore

Swagger Petstore

1.0.7 OAS 2.0

[Base URL: `petstore.swagger.io/v2`]
<https://petstore.swagger.io/v2/swagger.json>

This is a sample server Petstore server. You can find out more about Swagger at <http://swagger.io> or on [irc.freenode.net/#swagger](irc://irc.freenode.net/#swagger). For this sample, you can use the api key `special-key` to test the authorization filters.

[Terms of service](#)
[Contact the developer](#)
[Apache 2.0](#)
[Find out more about Swagger](#)

Schemes

HTTPS

Authorize

pet Everything about your Pets

Find out more

POST `/pet/{petId}/uploadImage` uploads an image

POST `/pet` Add a new pet to the store

PUT `/pet` Update an existing pet

GET `/pet/findByStatus` Finds Pets by status

GET `/pet/findByTags` Finds Pets by tags

GET `/pet/{petId}` Find pet by ID

POST `/pet/{petId}` Updates a pet in the store with form data

DELETE `/pet/{petId}` Deletes a pet

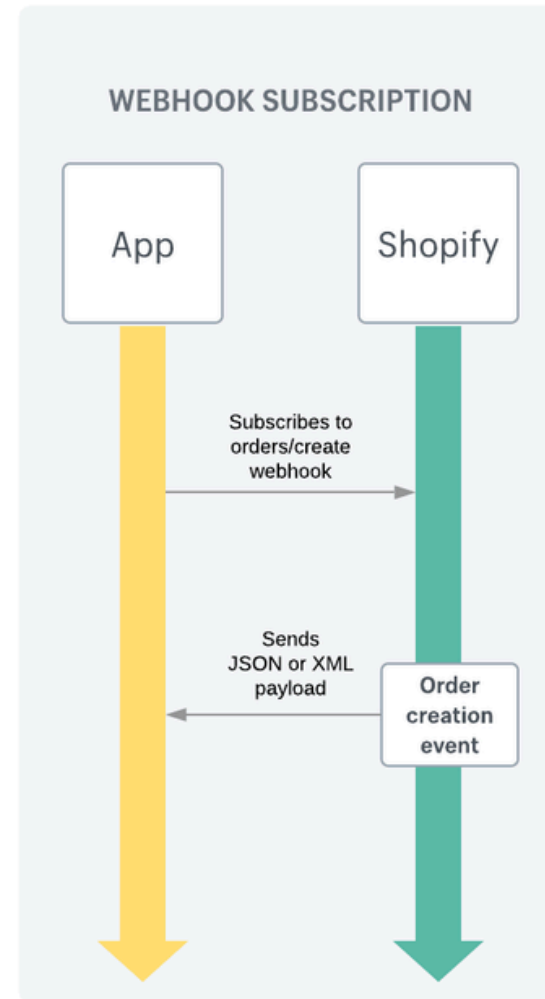
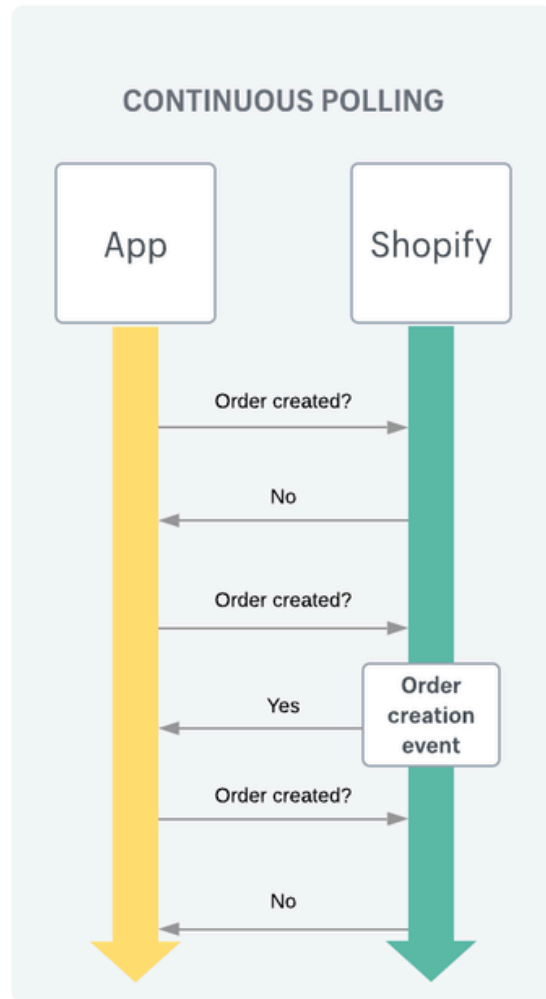
store Access to Petstore orders

GET `/store/inventory` Returns pet inventories by status

Webhooks

- REST segue o HTTP: cliente/servidor.
 - Como pode o cliente ser atualizado sobre um evento do lado do servidor?
 - Opção nº1: Polling a cada 5 segundos
 - Compromisso entre latência e sobrecarga do servidor.
 - Opção nº2: Callback!
 - O servidor atua temporariamente como um cliente HTTP, e o cliente como um servidor HTTP, para uma chamada inversa.
 - O callback precisa de ser registado previamente (webhook).
 - A API é definida pelo cliente do webhook (servidor) e não pelo servidor do webhook (cliente).

Webhooks



Autenticação em REST

- **HTTP Basic Auth**

- Password em plain-text. Não usar nunca.

- **API Keys**

- Cada cliente tem uma chave com que identifica todos os pedidos.
- Podem ser extraviadas. Devem ter uma lifetime curta.

- **Hash-based Message Authentication Code (HMAC)**

- Baseado em chave simétrica. Também deve ter lifetime curta.

- **OAuth 2**

- Usado para autenticação em serviços 3rd party.
- Tem granularidade a nível de acessos e conteúdos.
- É o default para Facebook, Google, etc...

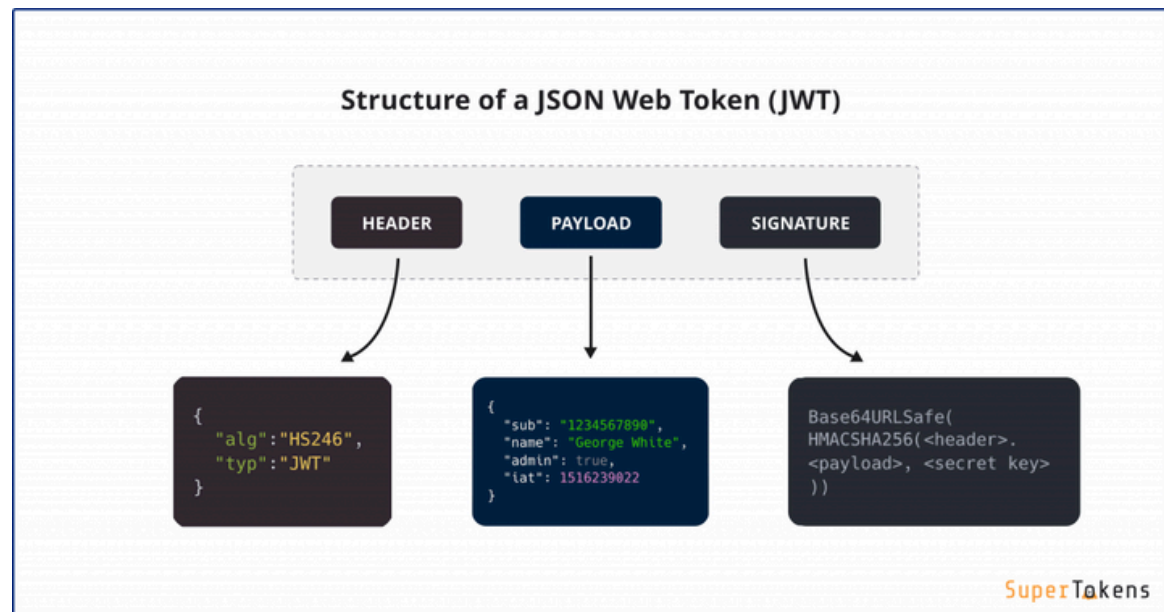
- **OpenID Connect**

- Baseado no OAuth2, mas focado apenas em identidade

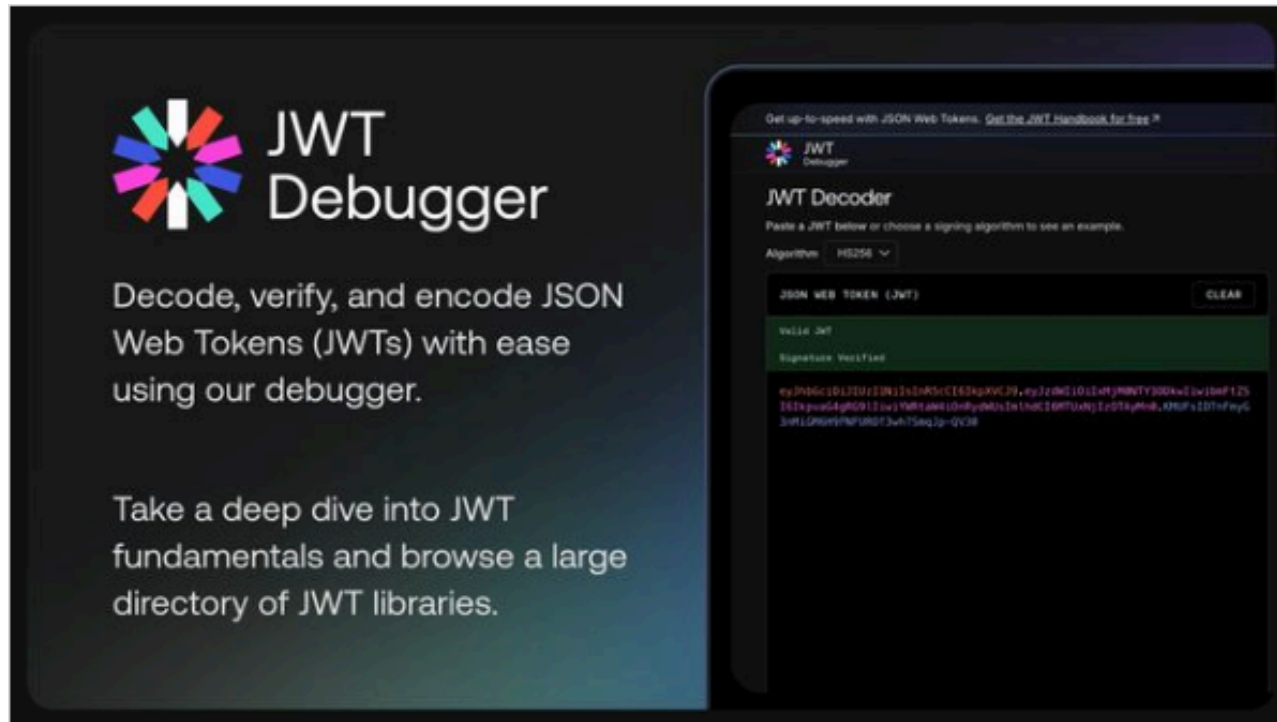
- **JWT**

JSON Web Token

- O servidor não precisa guardar sessão em banco de dados ou memória. A identidade e as permissões viajam dentro do próprio token.
- Header (com o algoritmo) → Payload → Assinatura
 - O Payload não é criptografado e não deve conter dados sensíveis.



JSON Web Token



JWT.IO

JSON Web Tokens are an open, industry standard RFC 7519 method for representing claims securely between two parties.

 [JSON Web Tokens /](#)

Alternativas?

- JSON é texto puro
 - Excelente para humanos → + banda +CPU (parsing)
 - Aumento de latência em redes lentas
- Tradução de JSON para modelos equivalentes internos.
 - DTOs em Java → outro padrão em Python, JS, Swift...
- Como fazer o upload de um ficheiro gigante em "partes" ou manter um chat sem fazer polling constante utilizando REST?

gRPC

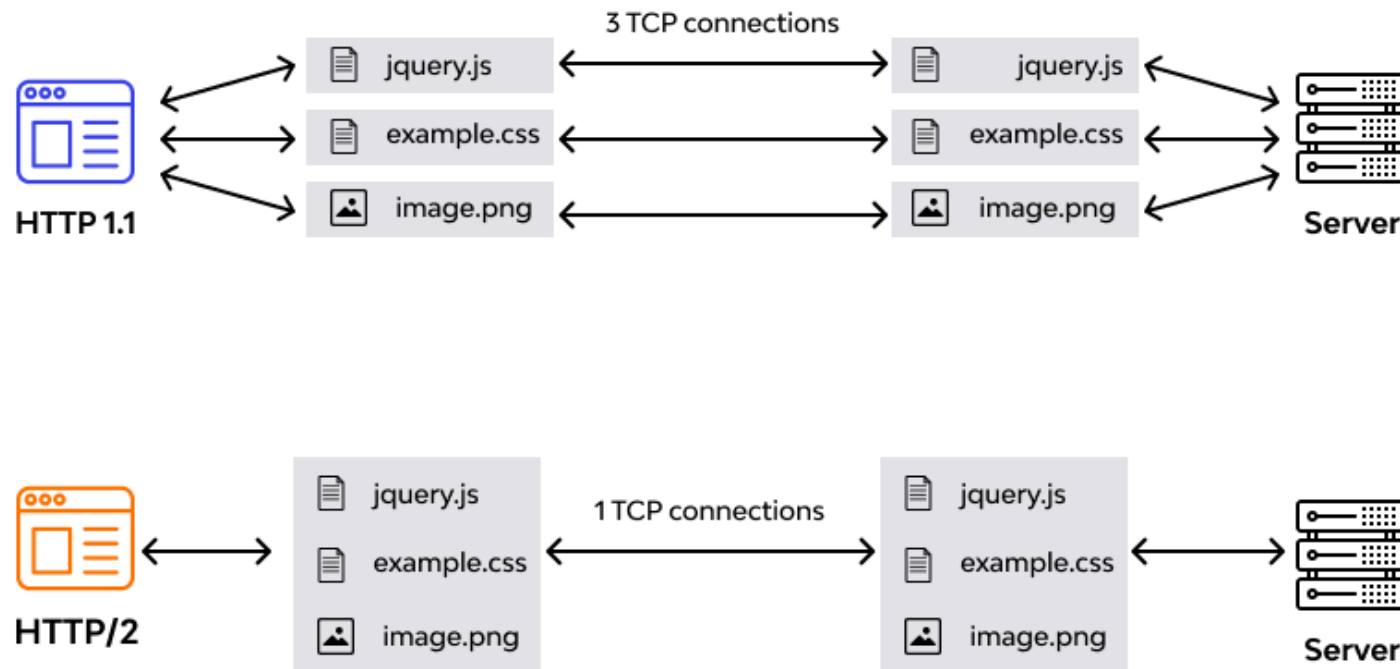
- Framework de alto desempenho para RPC (*Remote Procedure Call*)
 - Desenvolvido pela Google
- Sistema de serialização binária
 - +rápido -payload que JSON
 - Protocol Buffers (.proto)
- Utiliza HTTP para realizar a comunicação
- Permite mensagens do tipo **pedido-resposta** e **streaming**



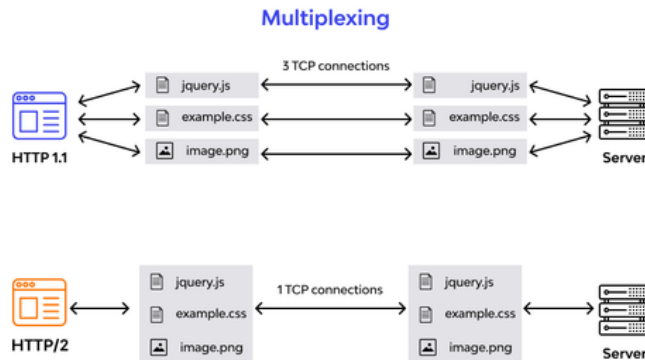
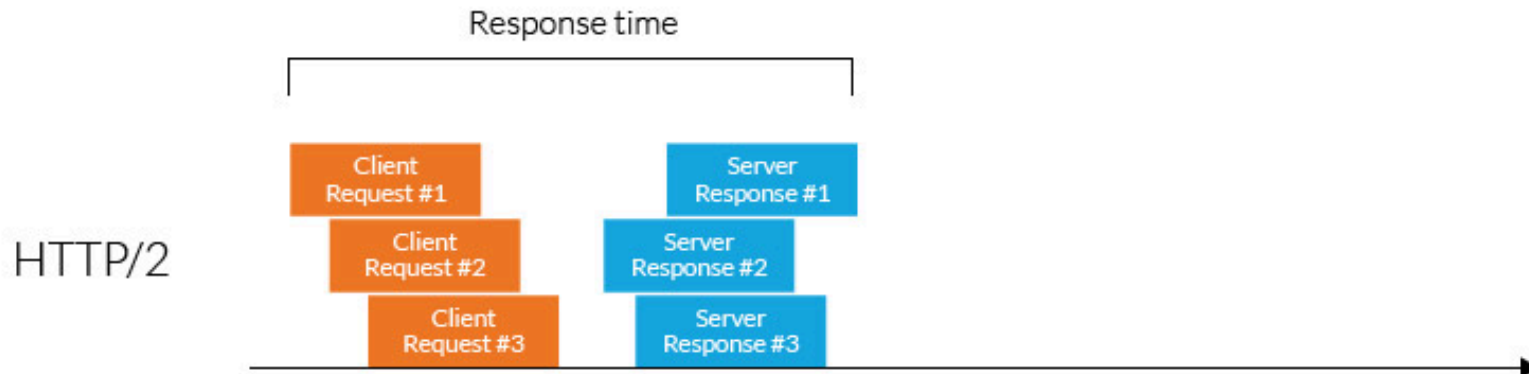
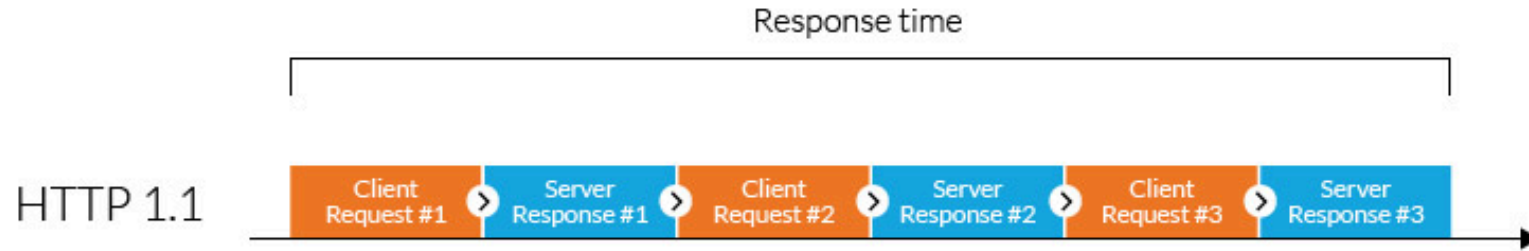
A high performance, open source
universal RPC framework

HTTP/1.1 vs HTTP/2

Multiplexing

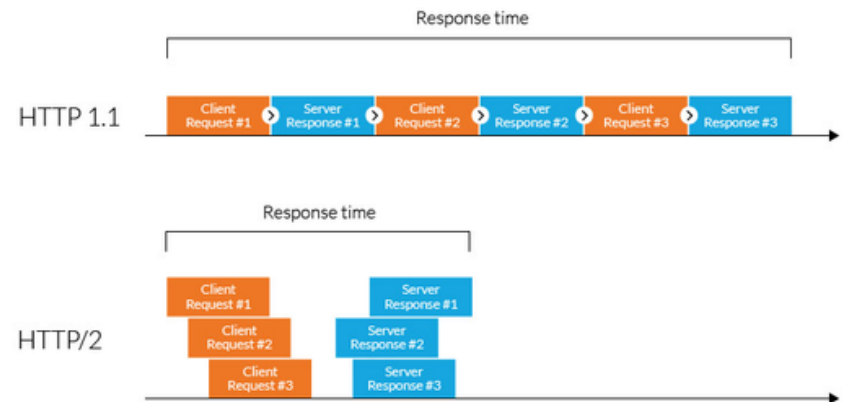
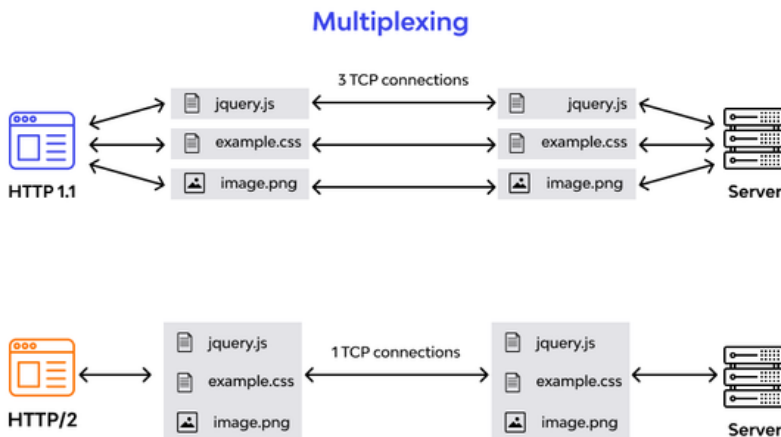


HTTP/1.1 vs HTTP/2



HTTP/1.1 vs HTTP/2

- **Multiplexação:** Permite múltiplas requisições e respostas simultâneas em uma única conexão TCP
- **Streaming bidirecional:** Dados enviados de forma contínua nas duas direções usando a mesma conexão.
- **Compressão de cabeçalhos:** - overhead e - volume de dados
- **Comunicação binária:** +eficiente e -erros



Protocol Buffers (ProtoBuf)

- Sistema de **serialização de dados estruturados**
 - Parecido com JSON ou XML
 - menor e +rápido
- Interfaces de serviços e estruturas de mensagens
- O compilador tem geração automática de código
 - Java, Kotlin, Python, C# /, C++, Dart, Go, Node ...

```
syntax = "proto3"; // ProtoBuf versão 3

package library;

message Book {
    int32 id = 1;
    string title = 2;
    string author = 3;
    int32 year_published = 4;
}

message Empty {}

message ListBooksResponse {
    repeated Book books = 1; //'repeated' → array
}

service LibraryService {
    rpc ListBooks (Empty) returns (ListBooksResponse);
    rpc AddBook (Book) returns (Book);

    // Método de streaming para enviar livros continuamente
    rpc StreamBooks (Empty) returns (stream Book);
}
```

Protocol Buffers (ProtoBuf)

```
service ProductService {  
  rpc CreateProduct(CreateProductRequest) returns (ProductResponse);  
  rpc GetProduct(GetProductRequest) returns (ProductResponse);  
  rpc UpdateStock(UpdateStockRequest) returns (ProductResponse);  
  rpc DeleteProduct(DeleteProductRequest) returns (Empty);  
}  
  
service CustomerService {  
  rpc CreateCustomer(CreateCustomerRequest) returns (CustomerResponse);  
  rpc GetCustomerByVat(GetCustomerByVatRequest) returns (CustomerResponse);  
  rpc GetAllCustomers(Empty) returns (GetAllCustomersResponse);  
}  
  
service SaleService {  
  rpc CreateSale(CreateSaleRequest) returns (SaleResponse);  
  rpc GetSaleById(GetSaleByIdRequest) returns (SaleResponse);  
  rpc GetAllSales(Empty) returns (GetAllSalesResponse);  
  rpc GetSaleProductsByVat(GetSaleProductsByVatRequest) returns (GetSaleProductsByVatResponse);  
  rpc AddProductToSale(AddProductToSaleRequest) returns (SaleResponse);  
  rpc CloseSale(CloseSaleRequest) returns (SaleResponse);  
  rpc UpdateSaleStatus(UpdateSaleStatusRequest) returns (SaleResponse);  
  rpc DeleteSale(DeleteSaleRequest) returns (Empty);  
}
```

Protocol Buffers (ProtoBuf)

```
service ProductService {
  rpc CreateProduct(CreateProductRequest) returns (ProductResponse);
  rpc GetProduct(GetProductRequest) returns (ProductResponse);
  rpc UpdateStock(UpdateStockRequest) returns (ProductResponse);
  rpc DeleteProduct(DeleteProductRequest) returns (Empty);
}

service CustomerService {
  rpc CreateCustomer(CreateCustomerRequest) returns (CustomerResponse);
  rpc GetCustomerByVat(GetCustomerByVatRequest) returns (CustomerResponse);
  rpc GetAllCustomers(GetAllCustomersRequest) returns (GetAllCustomersResponse);
}

service SaleService {
  rpc CreateSale(CreateSaleRequest) returns (SaleResponse);
  rpc GetSaleByVat(GetSaleByVatRequest) returns (SaleResponse);
  rpc GetAllSales(GetAllSalesRequest) returns (GetAllSalesResponse);
  rpc GetSaleProductsByVat(GetSaleProductsByVatRequest) returns (GetSaleProductsByVatResponse);
  rpc AddProductToSale(AddProductToSaleRequest) returns (SaleResponse);
  rpc CreateSale(CreateSaleRequest) returns (SaleResponse);
  rpc UpdateSaleStatus(UpdateSaleStatusRequest) returns (SaleResponse);
  rpc DeleteSale(DeleteSaleRequest) returns (Empty);
}
```

```
enum ProductDiscount {
  DISCOUNT_UNSPECIFIED = 0;
  ELIGIBLE = 1;
  NOT_ELIGIBLE = 2;
}
```

```
message ProductResponse {
  int64 id = 1;
  int32 code = 2;
  string description = 3;
  double face_value = 4;
  int32 stock_quantity = 5;
  ProductDiscount discount = 6;
}

message CreateProductRequest {
  int32 code = 1;
  string description = 2;
  double face_value = 3;
  int32 stock_quantity = 4;
  ProductDiscount discount = 5;
}

message GetProductRequest {
  oneof identifier {
    int64 id = 1;
    int32 code = 2;
  }
}

message UpdateStockRequest {
  oneof identifier {
    int64 id = 1;
    int32 code = 2;
  }
  int32 new_stock_quantity = 3;
}

message DeleteProductRequest {
  oneof identifier {
    int64 id = 1;
    int32 code = 2;
  }
}
```

```
message CustomerResponse {
  int64 id = 1;
  string name = 2;
  string vat = 3;
  string phone = 4;
}

message CreateCustomerRequest {
  string name = 1;
  string vat = 2;
  string phone = 3;
}

message GetCustomerByVatRequest {
  string vat = 1;
}

message GetAllCustomersResponse {
  repeated CustomerResponse customers = 1;
}
```

Comunicação com gRPC

- **Unary RPC:** pedido-resposta
 - `getUtilizador(id: 123)`
- **Server Streaming RPC:** Cliente faz um pedido e recebe um fluxo de respostas do servidor
 - `getNotificacoes(usuarioId)`
 - servidor envia notificações conforme elas chegam
- **Client Streaming RPC:** Cliente envia um fluxo de pedidos ao servidor que retorna uma resposta
 - Cliente envia chunks de um ficheiro grande
 - Servidor responde (ao final) "Upload Concluído"
- **Bidirectional Streaming RPC:** Cliente e servidor enviam fluxos de mensagens de forma independente e simultânea.
 - Chat de tempo real

Comunicação